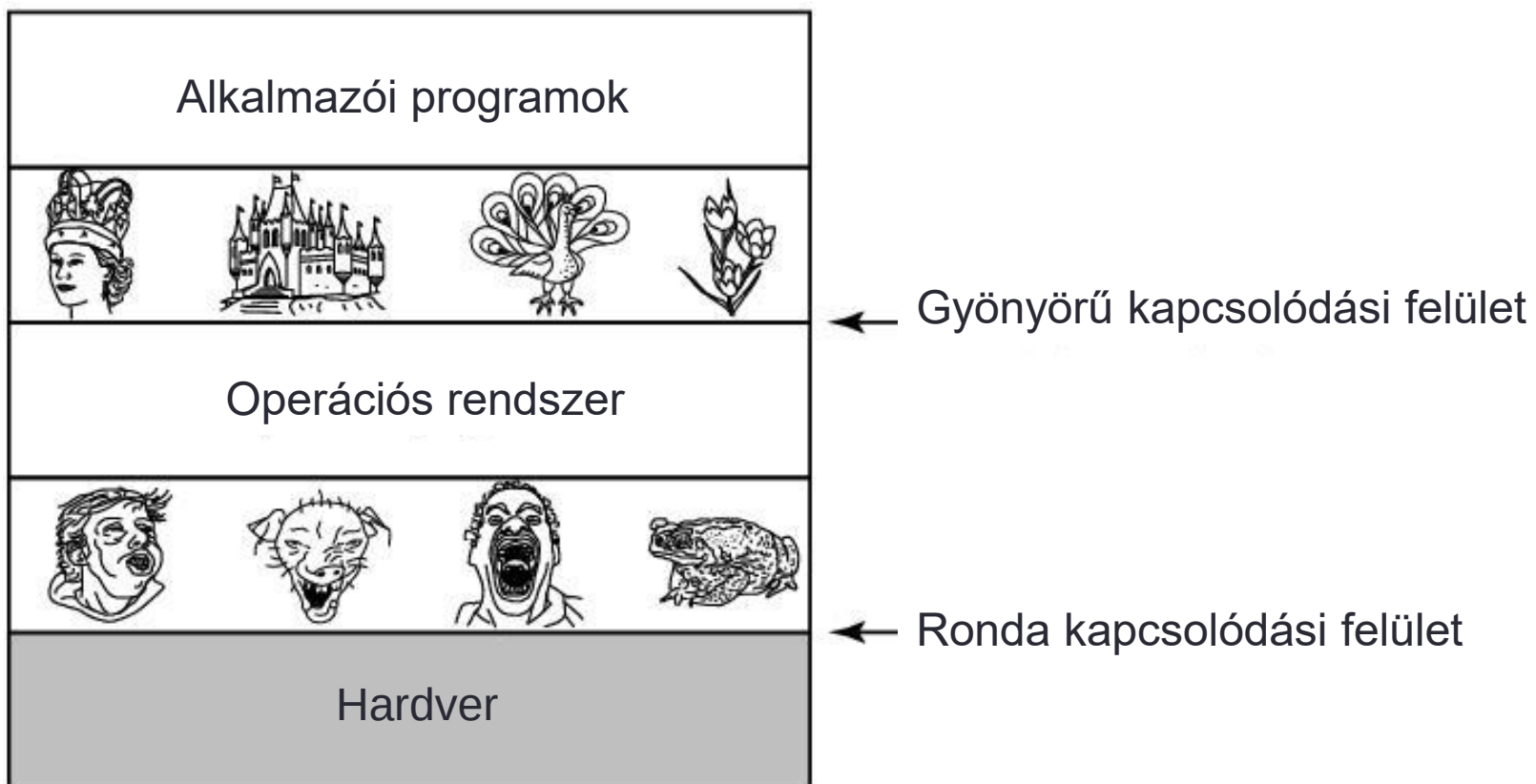


SZÁMÍTÓGÉPES ARCHITEKTÚRÁK

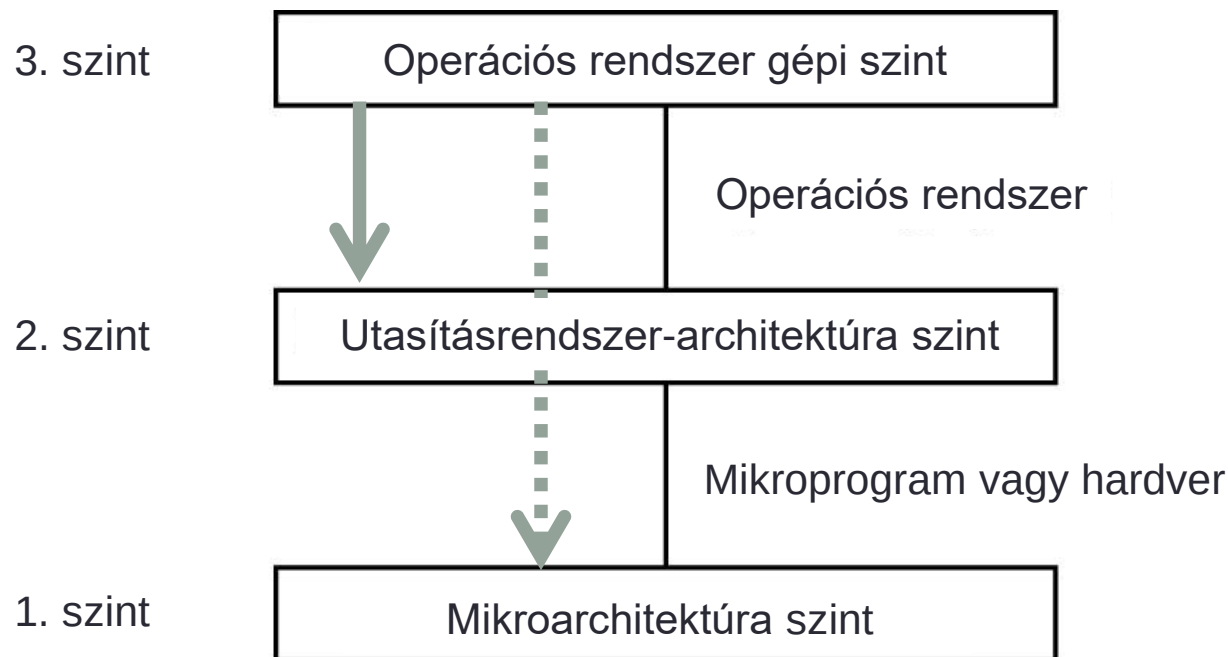
Az operációs rendszer gép szintje

Nagy Antal



Az operációs rendszer gépi szintje (OSM)

- Az ISA-szint felett és alatt
 - Új utasítások
- Szoftveres úton van megvalósítva
 - Hardveres megvalósításnak sincs akadálya



Az operációs rendszer gépi szintje (OSM)

- Nem igazi hardveres szintek
 - ISA
 - OSM
- Az OSM-szintű utasítások a teljes utasításkészletet jelentik
 - Az összes ISA-szintű utasítás
 - Operációs rendszer által hozzáadott **új utasítások**
 - Rendszerhívások (system calls)
- A rendszerhívások az OS valamely előre definiált szolgáltatását hívja meg
 - Pl. adatok olvasása fájlból

Az operációs rendszer gépi szintje

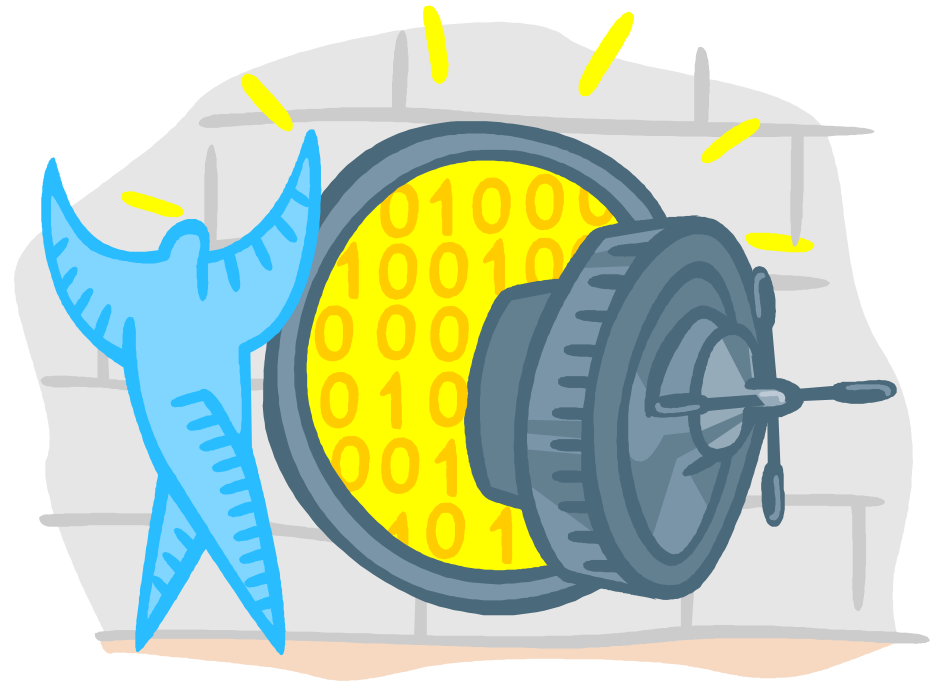
- OSM szint mindig interpretált
 - OSM utasítást lépésenként hajtja végre
- ISA-szintű utasítás végrehajtása
 - Az OS közreműködése nélkül
 - Közvetlenül az alatta lévő mikroarchitektúra szintjén végzi el
- Témakörök
 - Virtuális memória
 - Nagyobb memória elérése
 - Fájlszintű B/K
 - Párhuzamos feldolgozás
- Processzus
 - Futó program
 - Állapot információk
 - Memória
 - Regiszterek
 - Utasításszámláló
 - B/K állapot

Virtuális memória

- A memória kicsi és drága
 - Programozók feladata volt, hogy a programok beleférjenek az adott memória mennyiségbe
 - Lassabb programok a kis memória méret miatt
- Másodlagos memória
 - Lemez terület
- Programok felosztása kisebb részekre
 - Befértek a memóriába
 - Átfedések/Overlays
 - A részek külön-külön töltődtek be a memóriába
 - Gondoskodni kellett
 - Átfedésekről
 - Egyes részek elhelyezéséről a másodlagos memóriában
 - Memória és a lemez közötti mozgatusokról
 - ...

Virtuális memória

- Overlays
 - Sok munkával járt a kezelése
- Virtuális memória
 - Adminisztrációs feladatok elvégzése
 - 1960-s évek
 - Kutatási projektek
 - 1970-s évek
 - Legtöbb gépen elérhető volt

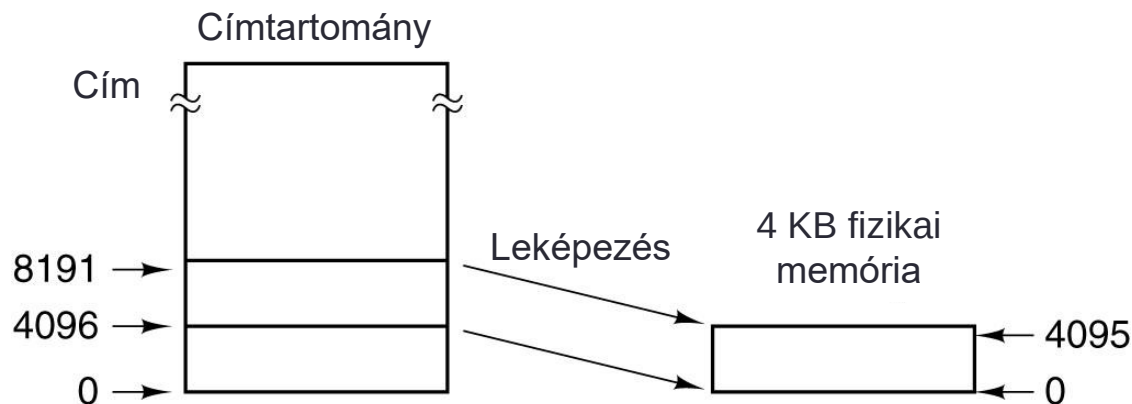


Lapozás

- Címtartomány és memóriarekeszek fogalmának a szétválasztása
 - 4096 szavas memória
 - Utasításokban 16 bites címrész
 - $2^{16}=65\ 536$ szót tud megcímezni
 - 4096 alatti és feletti címek megkülönböztetése
 - Hasznos és haszontalan címtartomány
 - A 4096 feletti címekhez nem tartozott valódi memória
 - A hardver egy-egy értelmű megfeleltetést hozott létre a címtartomány és a memóriarekeszek között

Lapozás

- A címtartomány és a memóriacímek elkülönítésének az elve
 - Bármely időpillanatban 4096 szó érhető el a memóriából
 - Ezek nem feltétlenül a 0 – 4095 közötti memóriacímeknek felelnek meg
 - Pl. a 4096 címre való hivatkozás esetén a memória 0-ás című szavát kell használni
 - A 4097-es cím esetén pedig a memória 1-es memóriacímét, stb.
 - Leképezés a címtartományból a valódi memória címek halmazára



Lapozás

- Egy 4KB-os, virtuális memória nélküli gép esetén
 - Fix leképezésünk van a 0-4095 közötti címek és a memória 4096 szava között
 - Mi történik, ha 8192 és 12287 közötti címre hivatkozunk?
 - Futási hibát kapunk
 - Nem létező memória címre való hivatkozás
- Virtuális memóriával rendelkező gép esetén a következő lépések hajtódnak végre
 1. A memória tartalmának a lemezre való mentése
 2. A 8192 és 12287 közötti szavak megkeresése a lemezen
 3. A lemezről való betöltése a 8192 és 12287 szavaknak
 4. A memória térkép megváltoztatása
 - A 8192 és 12287 közötti címek leképezése a 0 és 4095 közötti memóriarekeszre
 5. A végrehajtás folytatása

Lapozás

- Az átfedés automatikus kezelésének ezt a technikáját lapozásnak (paging) nevezzük
 - A lemezeiről beolvasott részeket pedig lapoknak (pages) nevezzük
- Virtuális címtartomány
 - Program által hivatkozott memóriacímek
- Fizikai címtartomány
 - Tényleges memória címek
- Memória térkép/laptábla
 - A virtuális címeknek megfelelő fizikai címek
- A programok a teljes virtuális címtartományt használhatják
 - A programozónak nem kell tudnia a háttértben folyó műveletekről
 - Transzparens mechanizmus

Lapozás

- A fizikai memória lehet nagyobb és kisebb is a virtuális címtartománynál
- Az operációs rendszer által szolgáltatott virtuális gép
 - Minden virtuális cím mögött valódi memória van
 - Illúzió
 - Csak az operációs rendszer íróinak kell tudnia, hogy hogyan tartható fent ez az illúzió



A lapozás megvalósítása

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

- Virtuális memória
 - Lemezterület
 - Lemezen lévő változat és a memóriába betöltött darabjai közötti frissítés
 - Ha változott a tartalom
- Azonos méretű **lapokra** való felosztása a virtuális címtartománynak
 - 2 valamely hatványa
- Fizikai címtartomány **lapkeretekre** való felosztás

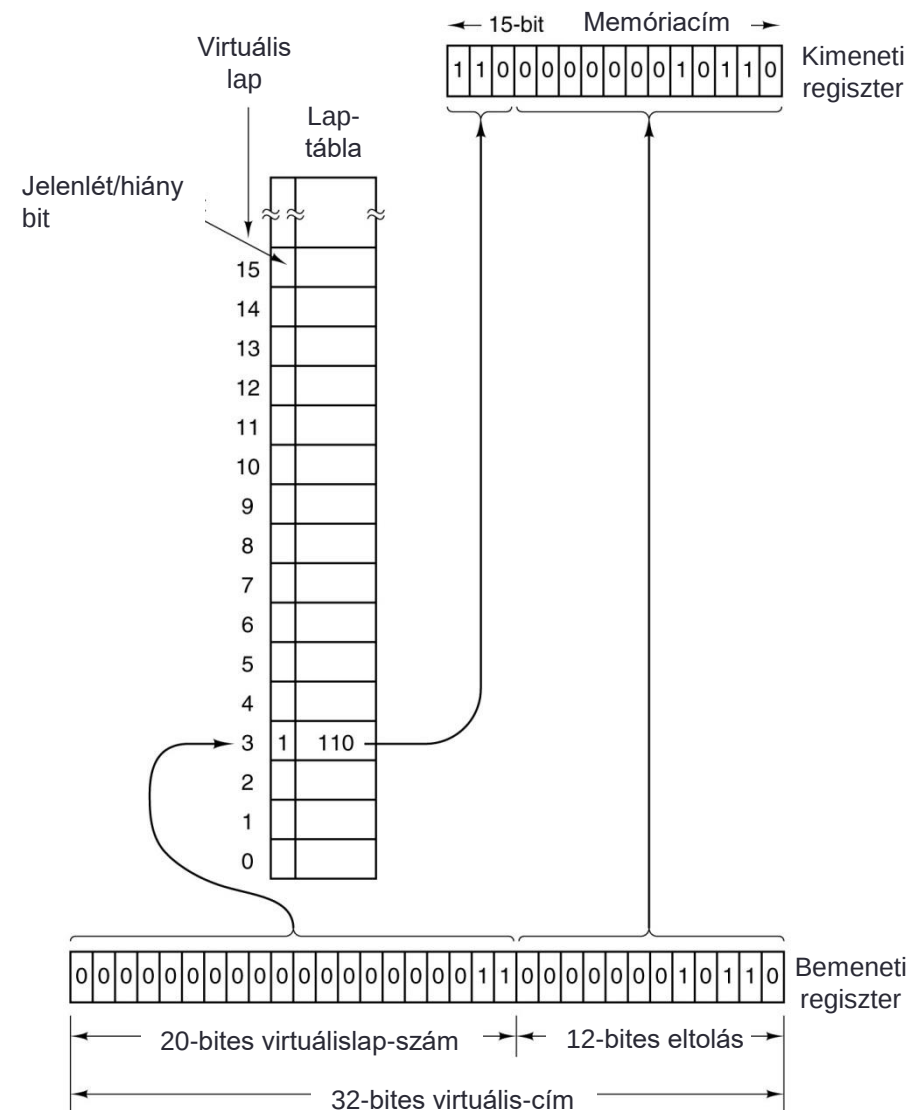
Lap	Virtuális címek
15	61440 – 65535
14	57344 – 61439
13	53248 – 57343
12	49152 – 53247
11	45056 – 49151
10	40960 – 45055
9	36864 – 40959
8	32768 – 36863
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

A fizikai memória alsó 32 KB-ja	
Lap-keret	Fizikai címek
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

A lapozás megvalósítása

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

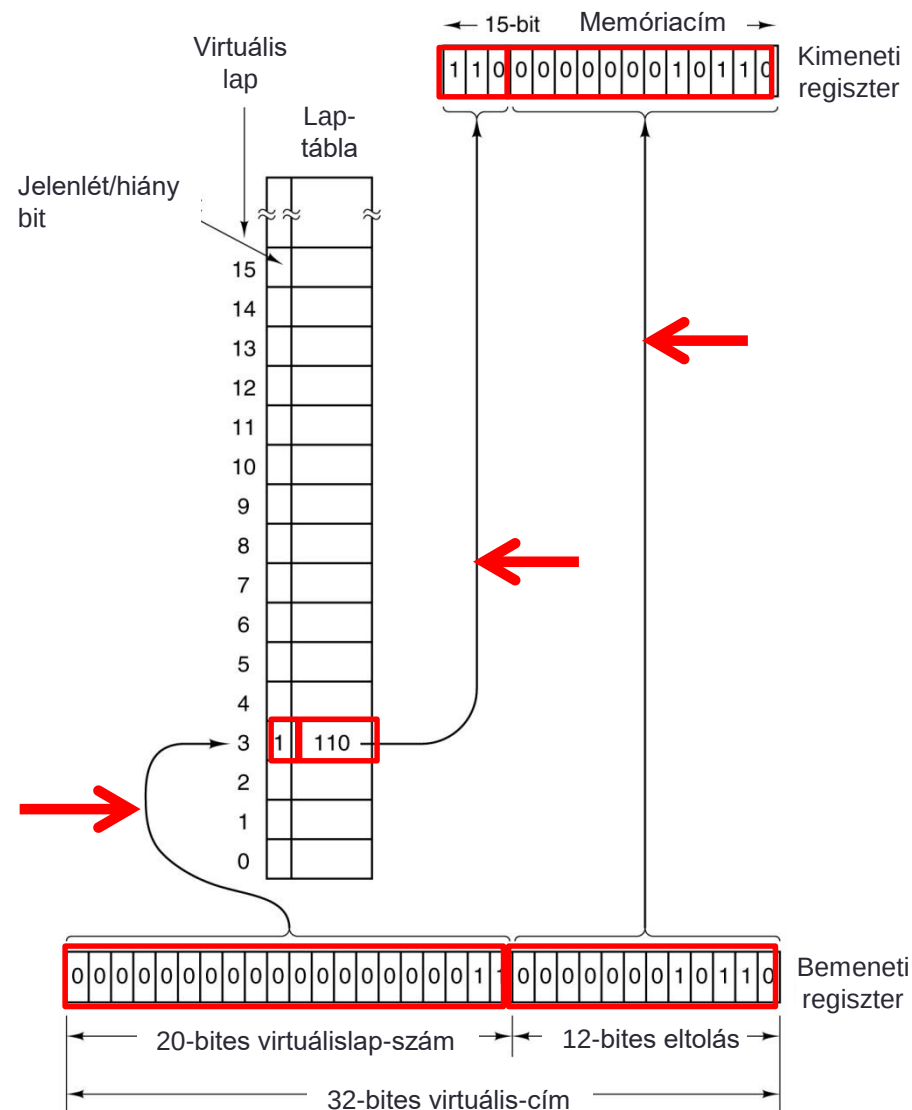
- Hogyan képezhető le egy 32 bites virtuális cím a fizikai memória egy címére?
 - Leképezést megvalósító egység
 - Memória kezelő egység
 - Memory Management Unit (MMU)
 - Vagy a CPU lapkán vagy külön helyezkedik el
 - 32 bites bemeneti regiszter
 - 15 bites kimeneti regiszter



A lapozás megvalósítása

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

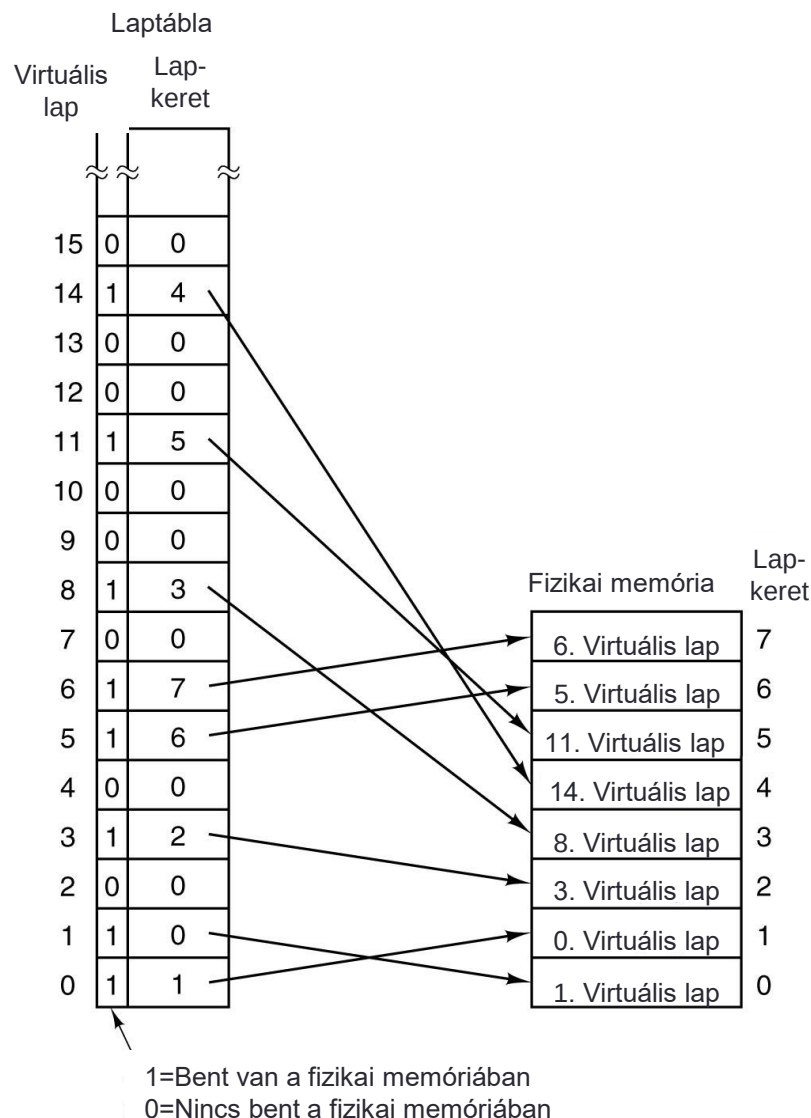
- 32 bites virtuális cím
 - 20 bites virtuális lapszám
 - 4 KB-os lapméret miatt
 - 12 bites eltolás (offset)
 - Lapszám → Index
 - MMU megvizsgálja, hogy a lap bent van-e a fizikai memóriában
 - Jelenlét/hiány bit
 - 3 bit átmásolása a 15 bites kimeneti regiszter felső 3 bitjére
 - Párhuzamosan az eltolás is átmásolódik az alsó 12 bitre



Lapozás megvalósítása

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

- Virtuális lapok egy lehetséges leképezése
- A leképezésnek gyorsnak kell lennie
- Jelenlét/hiány bit = 0
 - Page fault hiba
 - Kezelni kell





<https://live.voxvote.com>

PIN: 230173

A kérésre lapozás és a munkahalmaz modell

- A virtuális lap nem mindig található meg a fizikai memóriában
 - Nincs elég memória
- Laphiány (page fault) esetén az operációs rendszer feladatai
 - Be kell olvasni a lemezeről a kért lapot
 - Előfordulhat, hogy egy bent lévő lapot ki kell írni a lemezre előtte
 - Ki kell választani azt, hogy melyiket dobjuk ki
 - Be kell írnia az új helyét a laptáblába
 - Meg kell ismételni a hibát okozó utasítást

A kérésre lapozás és a munkahalmaz modell

- Amikor egy program egyetlen része sincs a memóriában
 - A laptáblában nincs bejegyzés
 - Indításkor a CPU az első utasítást megpróbálja betölteni
 - Egyből laphiány lép fel
 - Hatására az első lap betöltődik a memóriába
 - Másodlagos tárolóból
 - Az információ bejegyzésre kerül a laptáblában
 - Elkezdődhet az első utasítás végrehajtása
- Újabb laphiány léphet fel
 - Több is
- Ezt a fajta virtuális memória használatot kérésre lapozásnak (demand paging) nevezzük

A kérésre lapozás és a munkahalmaz modell

- A program elindulásakor lényeges, hogy a kérésre lapozást használjuk-e
 - Bizonyos idő elteltével a szükséges lapok bent lesznek a memóriában
- Időosztásos rendszernél
 - Több program van bent egyszerre a memóriában
 - Programváltáskor is változik a memória térkép
 - Kritikussá válhat a helyzet
 - Sok laphiány/page fault keletkezhet

A kérésre lapozás és a munkahalmaz modell

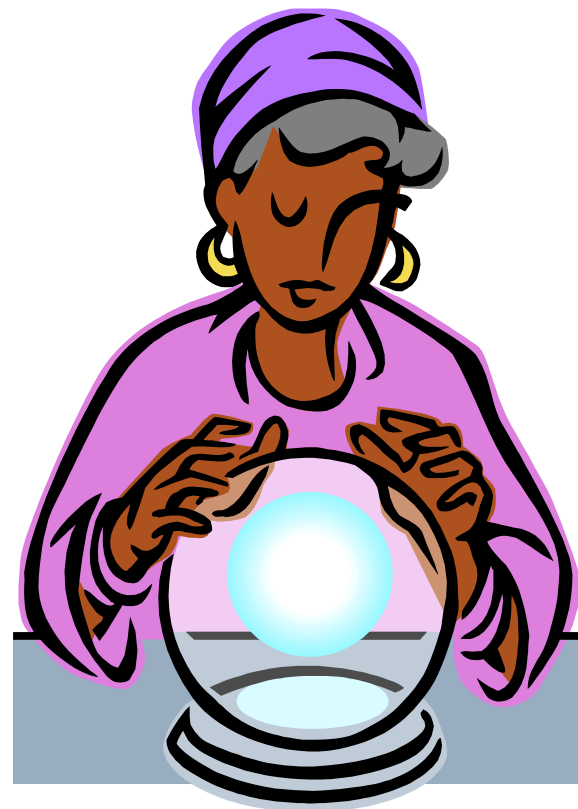
- A programok címtartományra való hivatkozása néhány lap körül sűrűsödnek össze
 - Lokalitás elv (locality principle)
- Egy memória hivatkozás
 - Utasítást tölthet be
 - Adatokat olvashat vagy írhat
- Munkahalmaz (working set)
 - Adott t időpillanatban a legutóbbi k memóriahivatkozásban szereplő lapok halmaza
 - Lassan változik
 - Becsülhető, hogy a program indításakor melyik lapokra van szükség
 - Betölthetőek az indítás előtt

Lapcserélő eljárások

- Ideális esetben a munkahalmazban lévő lapokat a memóriában lehetne tartani
 - Csökkenteni lehetne a laphiányok számát
 - Szimulációkat lehetne végezni
 - Ezek az eredmények csak az adott környezetben lennének érvényesek
 - A memória térkép állandóan változik
 - Az operációs rendszer feladata a munkahalmazhoz tartozó lapok felderítése
- Laphiány esetén a szükséges lapot be kell tölteni a lemezről a fizikai memóriába
 - Ha tele van a fizikai memória, akkor először helyet kell csinálni
 - Ki kell választani azt a lapot, amelyiket „kidobunk” a memóriából
 - Változás esetén ki kell írni a lemezre (virtuális memória)
 - Kell egy megfelelő algoritmus, ami jól választja ki a kidobandó lapot

Lapcserélő eljárások

- Ha olyan lapot dobunk ki, amelyikre nem sokkal későbbi utasításban hivatkozunk, akkor újabb laphiba keletkezik
- Az operációs rendszerek megpróbálják megjósolni
 - Leghaszontalanabb lapok kiválasztása
 - Eltávolítása minél későbbi időpontban fog laphiányt okozni
 - Minden lapra meg kellene határozni a várható legközelebbi hivatkozás időpontját
 - Azt kell kiválasztani, amelyiknél ez az érték a legnagyobb



Lapcserélő eljárások

- A legrégebben nem használt lap kiválasztása
 - Valószínűleg nincs benne a munkahalmazban
 - Least Recently Used (LRU)
 - Lapok listában vannak tárolva
 - Elől a legutoljára használt lap található
 - Minden memóriahivatkozáskor az adott lapot a lista elejére rakjuk
 - Hardveres implementáció
 - 64 bit-es számláló
 - Minden memóriahivatkozásnál automatikusan eggyel nő
 - Minden laptáblabejegyzésben van egy mező, amely az előző értéket tárolja
 - Laphiba esetén a legkisebb számlálóértékkel rendelkező lapot választjuk
 - Ez lesz a legrégebben használt lap



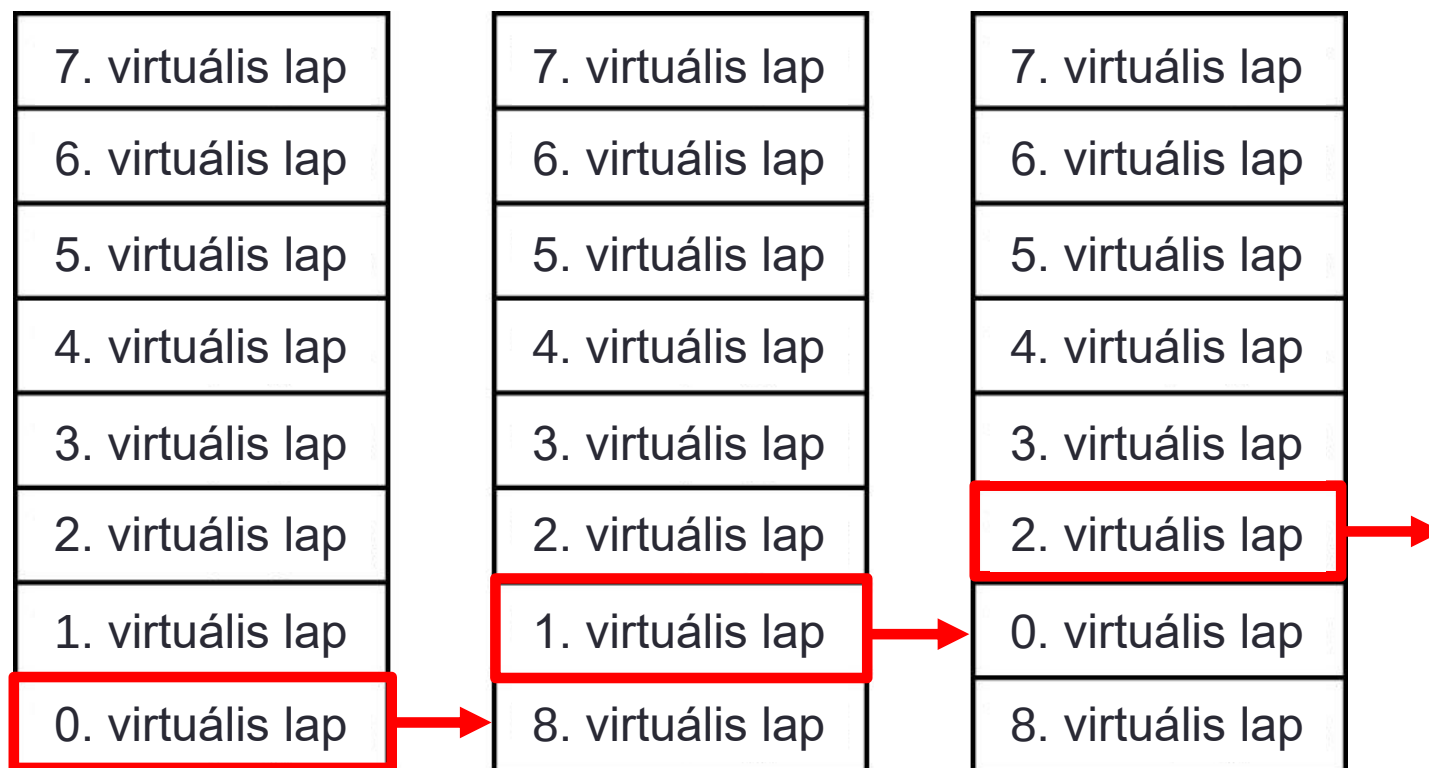
<https://live.voxvote.com>

PIN: 230173

Lapcserélő eljárások

- LRU Jól használható
 - Bizonyos esetekben nem működik jól

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0



Lapcserélő eljárások

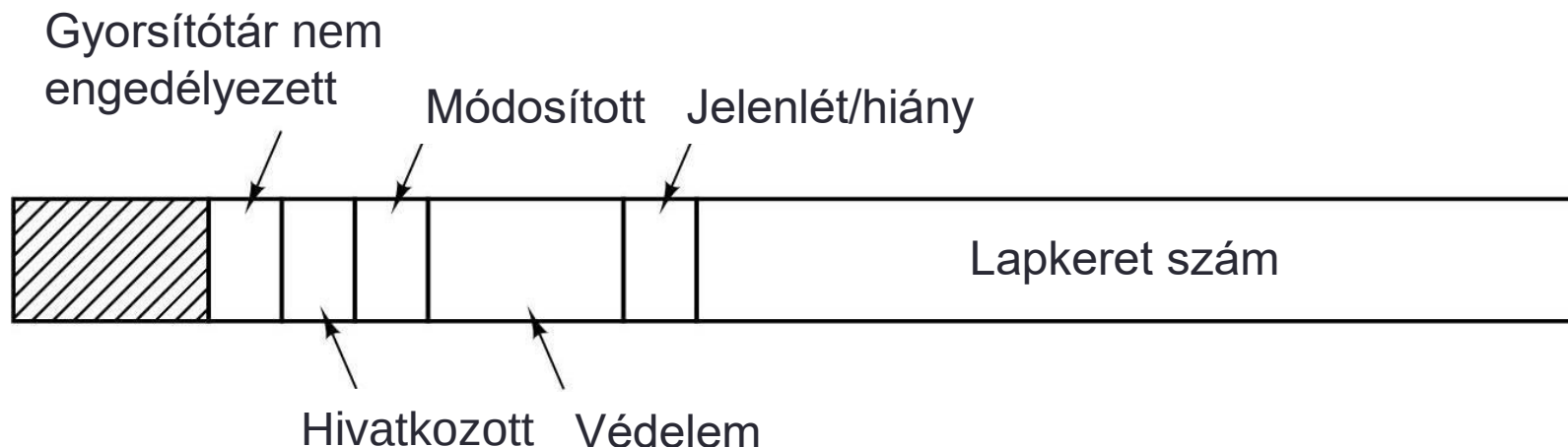
- Ha a rendelkezésre álló memória nagyobb a munkahalmazénál
 - Az LRU minimalizálja a laphiányok számát
- Másik algoritmus
 - FIFO (First In First Out)
 - A legrégebben betöltött lapot távolítja el
 - Nem számít az, hogy mikor hivatkoztak rá utoljára
 - Számláló, kezdetben 0
 - Eggyel növeljük a memóriában tartózkodó lapok számlálóját laphiánykor
 - Az új lapok számlálója 0
 - A legnagyobb számlálójú lapot választjuk ki

Lapcserélő eljárások

- Ha a munkahalmaz nagyobb, mint a rendelkezésre álló lapkeretek száma
 - Gyakori laphibák
 - Egyik algoritmus sem fog jó eredményt adni
 - Folyamatos laphiány
 - Vergődés
 - Nincs gond azokkal a programokkal
 - Nagy virtuális címtartományt használnak
 - Kicsi a munkahalmazuk
 - Időben lassan változó
 - Mindig elfér a rendelkezésre álló memóriában

Lapcserélő eljárások

- Ha a kiválasztott, „eldobandó” lapon nem történt módosítás
 - Nem kell visszaírni a lemezre
- Módosítás esetén frissíteni kell a lemez tartalmát
 - Virtuális memória lemezterület
- Laptábla bejegyzés



Lapméret és elaprózódás

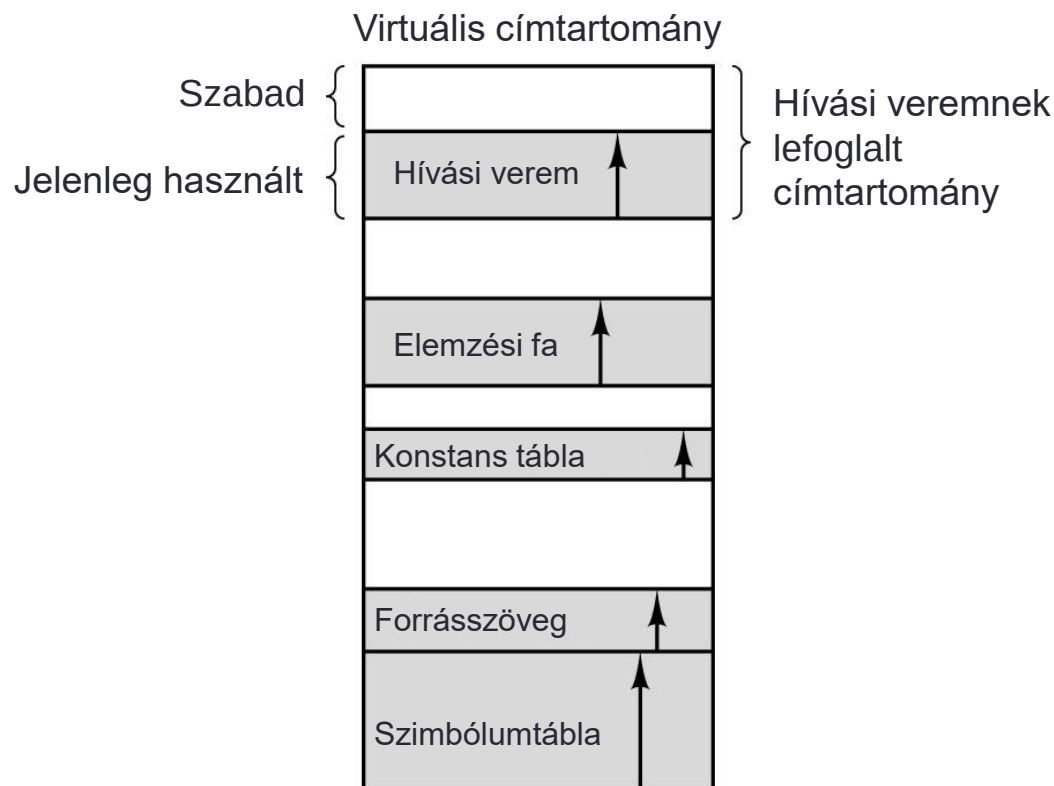
- A programokhoz tartozó utolsó lapon lehet kihasználatlan hely
 - Nem használja ki teljesen a lapméretet
 - Belső elaprózódás
- Ha lapméret n bájtos
 - Átlagosan $n/2$ lesz a veszteség
- Kis lapméretet választva
 - Sok lap
 - Nagyméretű laptábla
 - Hardveres megvalósításnál több regiszter
 - Drágább

Szegmentálás

- Az eddigi virtuális memória lineáris (egydimenziós)
 - A címek 0-tól kezdődtek
- Sok esetben előnyösebb több külön címtartomány alkalmazása
- Például egy fordító programnak a fordítás során több táblázatot használ
 - 1. Változók nevét és attribútumait tartalmazó szimbólum tábla
 - 2. A listázáshoz megőrzött forráskód
 - 3. Az összes felhasznált egész és lebegőpontos konstanst tartalmazó tábla
 - 4. A program szintaktikus elemzésekor létrehozott elemzési fa
 - 5. A fordítóprogramon belüli eljáráshíváshoz tartozó verem

Szegmentálás

- Az első négy tábla mérete folyamatosan nő
- A verem mérete nő illetve csökken
- Egydimenziós memória
- Kivételesen sok változó esetén problémák adódnak

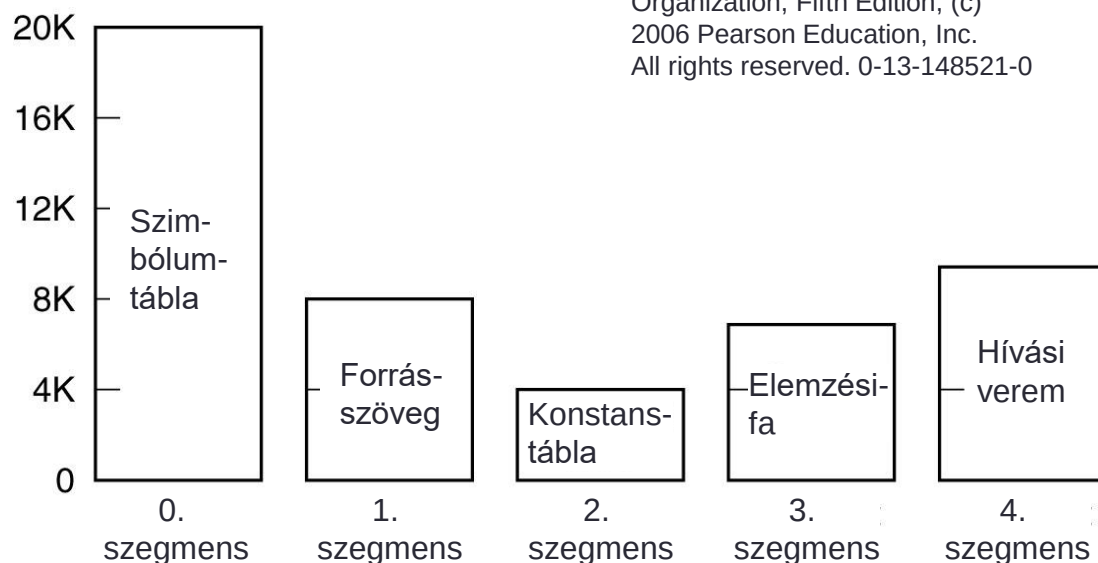


Szegmentálás

- Változó méretű táblák probléma

- Szegmensek

- Független címtartományok
 - 0-tól egy megadott méretig
 - Hosszuk változhat
 - Két dimenziós memória
 - Szegmens száma
 - Szegmensen belüli cím
 - Logikai egység
 - Programozó számára látható



Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0

Szegmentálás

- A szegmens tartalmazhat
 - Eljárást
 - Tömböt
 - Vermet
 - Skaláris változókból álló gyűjteményt
- Más előnyök
 - Különböző eljárások 0 kezdőcímtől másrés szegmensekben helyezkednek el
 - A külön lefordított eljárások könnyebben szerkeszthetőek össze
 - Megcímezése (n,0) címmel történik
 - Ha módosul az egyik eljárás, akkor a többit nem kell újrarendíteni
 - Különböző programok közötti közösen használt kód- és adatmegosztás

Szegmentálás

- Szegmensek védelme
 - Eljárás szegmensek, csak végrehajthatóak
 - Tilos olvasni és felülni
 - Lebegőpontos tömbök
 - Írhatóak, olvashatóak
 - Nem végrehajthatóak
 - Segíti a programozási hibák megtalálását
- Tudjuk, hogy mi van eltárolva az adott szegmensekben
- A lineáris esetben nem így van
 - Lapok tartalma bizonyos értelemben véletlenszerű
 - A programozó nem tud a lapozásról

Összehasonlítás

Szemponatok	Lapozás	Szegmentálás
Tudnia kell-e róla a programozónak?	Nem	Igen
Hány lineáris címtartomány létezik?	1	Sok
Meghaladhatja-e a virtuális címtartomány nagysága a fizikai memória méretét?	Igen	Igen
Könnyen kezelhetők-e a változó méretű táblák?	Nem	Igen
Milyen céllal dolgozták ki ezt a technikát?	Nagy memória szimulálása	Több címtartomány biztosítása



<https://live.voxvote.com>

PIN: 230173

A szegmentálás megvalósítása

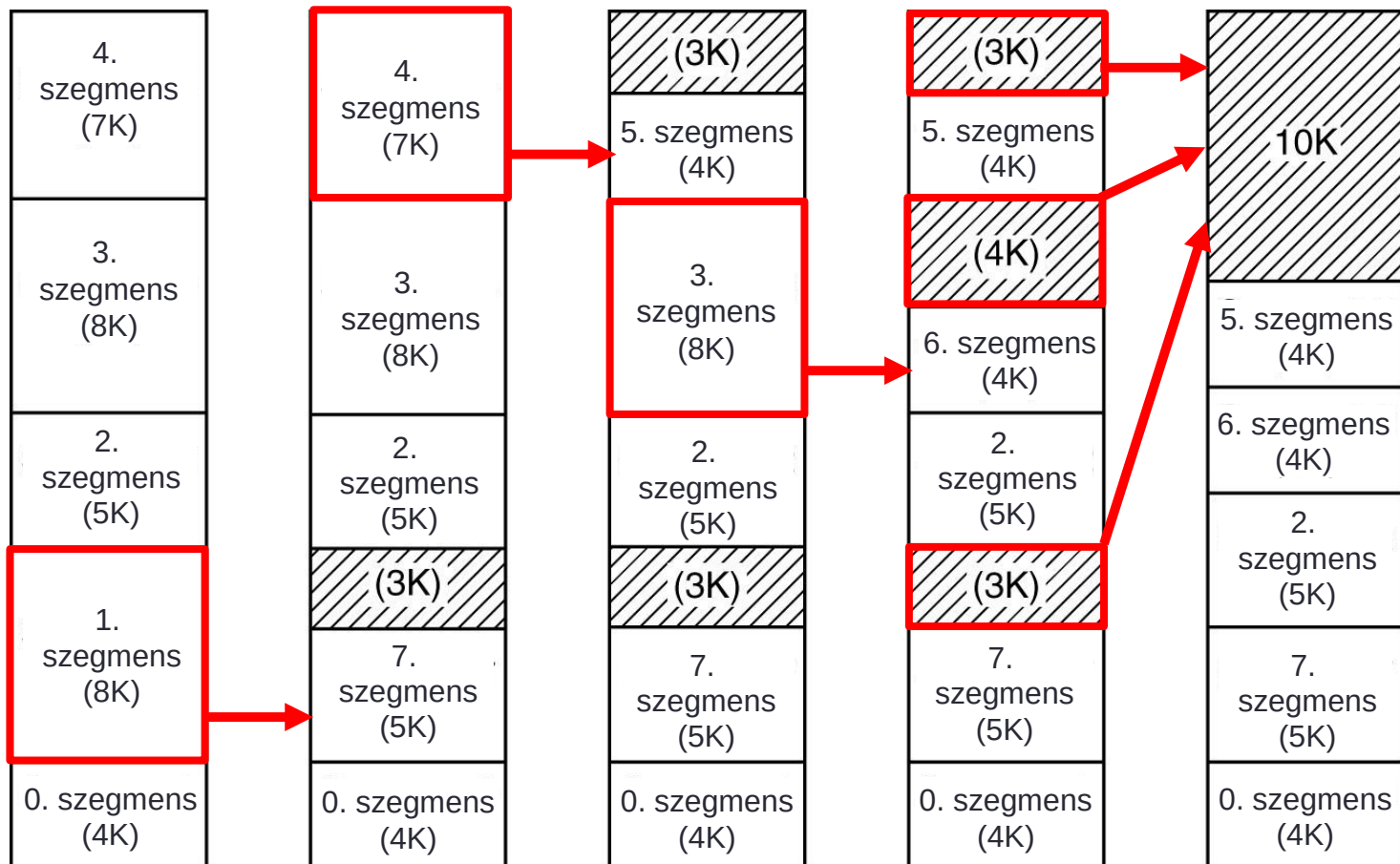
- **Csere (swapping)**

- Minden időpillanatban a szegmensek bizonyos halmaza van a memóriában
- Ha olyan szegmensre hivatkozunk, ami nincs bent a memóriában
 - Betöltődik
- Ha nincs elég hely
 - Egy vagy több szegmenst ki kell írni
 - Ha tiszta, akkor eldobható
- Hasonlít a kérésre való lapozáshoz
 - A szegmensek mérete változó

A szegmentálás megvalósítása

- Példa változó méretű szegmens cseréjére

Forrás: Tanenbaum, Structured Computer Organization, Fifth Edition, (c) 2006 Pearson Education, Inc. All rights reserved. 0-13-148521-0



Szegmentálás megvalósítása

- Memória szegmensek és lyukak
 - Külső elaprózódás
- Előfordulhat, hogy az elaprózódás miatt nem tölthető be egy szegmens
 - Hiába van összességében elegendő memória
 - Pl. a 3. szegmensre hivatkozva az előző példában
 - Mérete 8K
 - Összességében 10K szabad terület van
 - Nem egybefüggő
 - 3 részben
- Elaprózódás elkerülése
 - 0. memóriacím felé toljuk a lyuk mögötti szegmenseket
 - Túl sok időt igényel
 - Megvárjuk, amíg a külső elaprózódás egy „komolyabb” szintet el nem ér
 - Csak ezután rendezzük át a memóriát

Szegmentálás megvalósítása

- Algoritmusok, amelyek meghatározzák, hogy az adott szegmenseket melyik üres helyre töltsük be
 - Szükség van
 - Lyukakat tartalmazó lista
 - Cím
 - Méret
 - Legjobb illeszkedés
 - A legkisebb üres terület kiválasztása, amelyikbe még belefér a szegmens
 - Nagy lyukak „kímélése”
 - Nem darabolódnak fel
 - Kicsi üres területet keletkeznek, amit nem lehet később már felhasználni
 - Első illeszkedés
 - A listán körbe megy
 - Az első üres terület kiválasztása, amely elegendően nagy a szegmens beolvasásához
 - Általános hatékonyság szempontjából jobb
 - Gyorsabb

Szegmentálás megvalósítása

- Mindkét algoritmusnál csökken a lyukak átlagos nagysága
 - Az idő múlásával apró, „haszontalan” üres területek keletkeznek
 - Szükség van kis üres területek összevonására
 - Szegmens eltávolításakor
 - Két szomszédos üres terület összevonásával nagyobb üres területet hozhatunk létre

Szegmentálás megvalósítása

- **Lapozás**

- Szegmensek fix méretű lapokra való felosztása
- Előfordulhat, hogy valamely szegmens lapjainak egy része a memóriában van a többi meg a lemezen van tárolva
- Lapozáskor minden szegmensnek külön laptábla kell
- A szegmenseken belül egy lineáris címtartományt használunk
 - A korábbi lapozási technikák használhatóak

- **MULTICS**

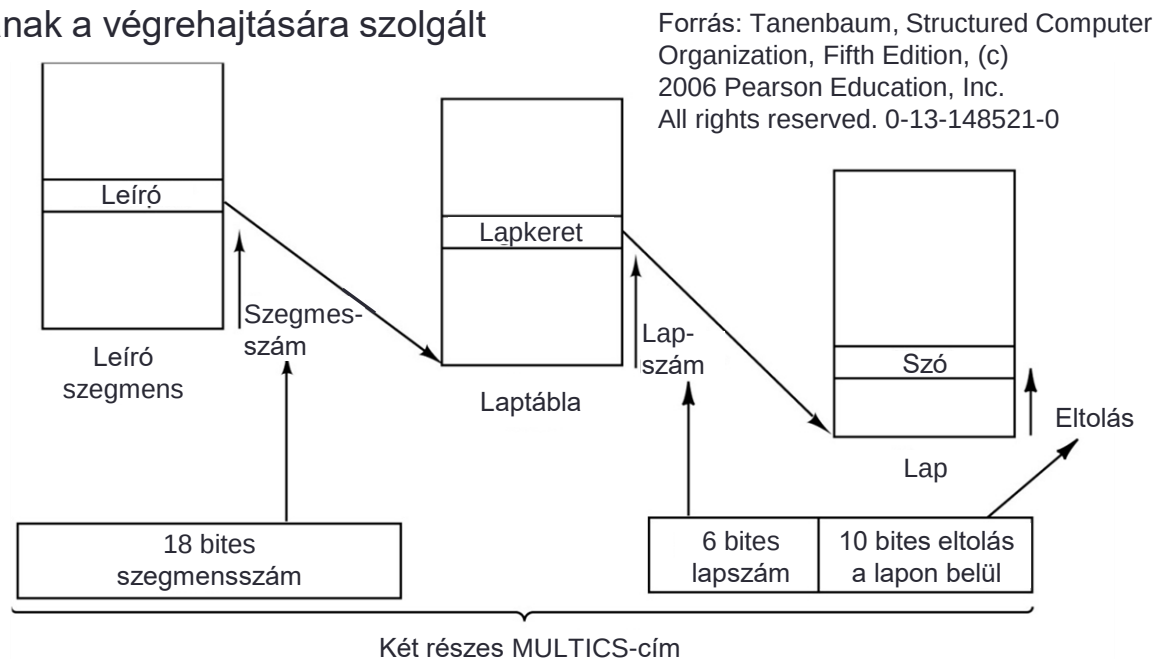
(MULTiplexed Information and Computing Service)

- Lapozással kombinált szegmentálás használt
- A címek két részből álltak
 - Szegmensszámból
 - Szegmens belüli címből

Szegmentálás megvalósítása

• MULTICS

- Minden processzushoz tartozott egy leíró szegmens
 - Szegmens leírók
 - Amikor a hardver egy virtuális címet kapott
 - Indexként használta és kikereste a megcímzendő szegmens leírót
 - A leíró egy szegmens lapjára mutatott
 - Szegmensek lapozásának a végrehajtására szolgált
 - Asszociatív memória
 - Gyorsítás
 - Legutóbb használt szegmens/lap kombinációja
- Intel 386-os óta
 - Virtuális memóriakezelés



Virtuális memória és gyorsítótár

Virtuális memória

- A program a lemezen van
 - Fix méretű lapokra van felosztva
 - A lapok egy részhalmaza van a memóriában
 - Kevés laphiány esetén a futása gyors lesz
- Különbségek
 - Laphiányokat
 - Operációs rendszer kezeli
 - A virtuális címek legnagyobb helyértékű bitje szerint szervezik a laptáblákat

Gyorsítótár

- Az egész program a memóriában van
 - Fix méretű gyorsítóblokkok
 - A program többnyire a gyorsítótárban lévő blokkokat használja
 - Kevés gyorsítótárhiány esetén a futása gyors lesz
- Különbségek
 - Gyorsítótárhiányokat
 - Hardver kezeli
 - Mérete kisebb, mint a lapméret
 - 64 bájt illetve 8 KB
 - A memória címek legkisebb helyértékei szerint indexelnek

Implementációs eltérések, de hasonló alapelvek



<https://live.voxvote.com>

PIN: 230173